

Segmentation, Paging and Segmentation with Paging

Fundamentals of Operating System Design

Prashant Jadav

DEPSTAR

Learning Objectives

By the end of this lecture, students should be able to:

- Explain segmentation and perform segment-address translation.
- Explain paging and perform page-number/frame-number translation.
- Compare internal and external fragmentation.
- Explain segmentation with paging and its address format.
- Solve numerical problems involving page tables, segment tables and address translation.
- Analyze advantages and disadvantages of each memory-management scheme.
- Solve representative GATE previous-year questions (PYQs).

Big Picture: Three Memory-Management Schemes

Segmentation
Variable-size logical units

Main issue:
external fragmentation

Paging
Fixed-size pages

Main issue:
page-table overhead + internal fragmentation

Segmentation + Paging
Logical units + fixed pages

Combines
benefits,
adds translation overhead

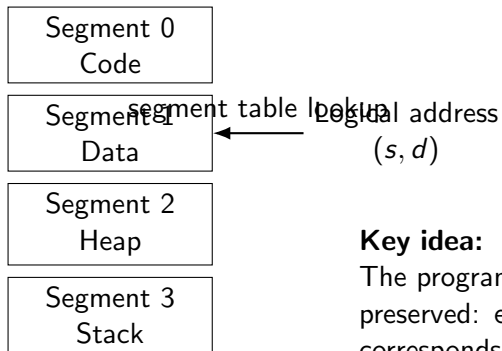
Segmentation: Basic Idea

- A process is divided into **logical segments**.
- A segment represents a meaningful program unit:
 - code / text
 - data
 - stack
 - heap
 - shared library
- Segments can have different sizes.
- A logical address is:

⟨segment number, offset⟩

- Each segment has a **base** and a **limit**.

Segmentation: Logical View



Key idea:

The programmer's view is preserved: each segment corresponds to a logical entity.

Segment Table

A segment-table entry typically contains:

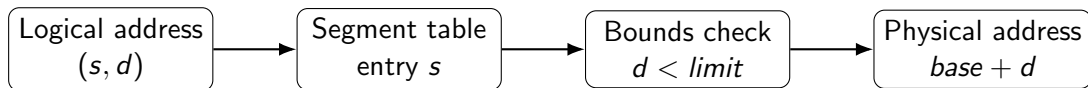
Field	Meaning
Base	Starting physical address of segment
Limit	Length / maximum valid offset
Protection	Read/write/execute permissions
Valid/Present	Whether the segment is valid/present
Other bits	Sharing, privilege, status, etc.

For a logical address (s, d) :

$$\text{If } d < \text{limit}[s], \quad PA = \text{base}[s] + d$$

Otherwise, the reference causes a protection/bounds fault.

Segmentation Address Translation



$$PA = Base_s + Offset$$

provided

$$Offset < Limit_s$$

Segmentation: Numerical Example 1

Given:

- Segment 2 has base address 12000.
- Segment 2 has limit 4000 bytes.
- Logical address = (2, 3500).

Step 1: Bounds check

$$3500 < 4000 \Rightarrow \text{valid}$$

Step 2: Translation

$$PA = 12000 + 3500 = \boxed{15500}$$

Exam trap: Do not add the segment number to the offset. The segment number selects the segment-table entry.

Segmentation: Numerical Example 2 – Fault

Suppose:

$$Base_3 = 50000, \quad Limit_3 = 6000$$

and logical address is:

$$(3, 6200)$$

Bounds check:

$$6200 < 6000 \quad \text{is false}$$

Therefore:

Invalid reference / segmentation fault

Even though

$$50000 + 6200 = 56200$$

can be numerically calculated, the address is **not legally accessible**.

Important

The limit is a protection boundary, not merely a value used after addition.

Advantages and Disadvantages of Segmentation

Advantages

- Matches programmer's logical view.
- Natural protection per segment.
- Convenient sharing of code/data segments.
- Segments can grow independently.
- Supports meaningful access-control boundaries.

Disadvantages

- Variable-size allocation causes external fragmentation.
- Compaction may be required.
- Allocation is more complex than fixed-size paging.
- Segment-table management adds overhead.
- Large segments may be difficult to place in fragmented memory.

Paging: Basic Idea

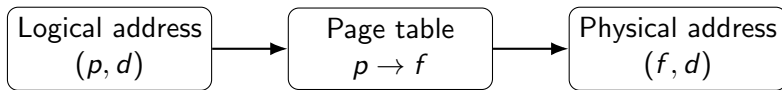
- Physical memory is divided into fixed-size **frames**.
- Logical/virtual memory is divided into fixed-size **pages**.
- Page size = frame size.
- Pages of a process can be placed in any available frames.
- Logical address:

$\langle \text{page number}, \text{offset} \rangle$

- Page table maps:

$\text{page number} \rightarrow \text{frame number}$

Paging: Address Translation



If page size is 2^k bytes:

$$\text{offset bits} = k$$

For a n -bit logical address:

$$\text{page-number bits} = n - k$$

Paging: Numerical Example 1

Given:

- Logical address = 16 bits.
- Page size = 1 KB = 2^{10} bytes.
- Page table maps page 5 to frame 12.
- Logical address = (5, 100).

Number of offset bits:

$$\log_2(1024) = 10$$

Number of page-number bits:

$$16 - 10 = 6$$

Translation:

$$(p, d) = (5, 100) \rightarrow (f, d) = (12, 100)$$

Physical address:

$$12 \times 1024 + 100 = \boxed{12388}$$

Paging: Numerical Example 2 – Bit Fields

Consider a 32-bit logical address and page size:

$$4 \text{ KB} = 2^{12} \text{ bytes}$$

Therefore:

$$\text{Offset} = 12 \text{ bits}$$

Remaining bits:

$$32 - 12 = 20$$

Hence:

$$\text{Page number} = 20 \text{ bits}$$

Number of virtual pages:

$$2^{20}$$

If physical memory is 2^{20} bytes:

$$\text{Frame number bits} = 20 - 12 = 8$$

Paging: Page Table Size

If:

Logical address = n bits

Page size = 2^k bytes

PTE size = e bytes

Then:

Number of pages = 2^{n-k}

Single-level page-table size:

$$2^{n-k} \times e \text{ bytes}$$

Example

32-bit VA, 4 KB pages, 4-byte PTE:

$$2^{32-12} \times 4 = 2^{20} \times 4 = 4 \text{ MB}$$

Advantages and Disadvantages of Paging

Advantages

- Eliminates external fragmentation.
- No need for contiguous allocation.
- Easy allocation of fixed-size frames.
- Supports virtual memory naturally.
- Efficient swapping/page replacement.

Disadvantages

- Internal fragmentation can occur.
- Page tables consume memory.
- Address translation adds overhead.
- TLB/cache hierarchy is needed for performance.
- Programmer's logical view is not directly represented.

Internal vs External Fragmentation

Internal fragmentation

- Wasted space **inside** an allocated unit.
- Common in paging.
- Last page of a process may be partially unused.

Paging $\Rightarrow$ internal fragmentation

External fragmentation

- Free space exists but is split into separated holes.
- Common in variable-size allocation/segmentation.
- Compaction can reduce it.

Segmentation $\Rightarrow$ external fragmentation

These are tendencies, not absolute statements about every implementation.

Paging: Internal Fragmentation Example

Suppose:

Process size = 18.5 KB

Page size = 4 KB

Pages required:

$$\left\lceil \frac{18.5}{4} \right\rceil = 5$$

Allocated memory:

$$5 \times 4 = 20 \text{ KB}$$

Internal fragmentation:

$$20 - 18.5 = \boxed{1.5 \text{ KB}}$$

Exam insight

For a single process, the unused space in the last page is less than one page.

Segmentation vs Paging: Conceptual Comparison

Property	Segmentation	Paging
Unit	Variable-size segment	Fixed-size page
Physical unit	Variable-size region	Fixed-size frame
Address	(s, d)	(p, d)
Mapping	Segment table	Page table
Fragmentation	External	Internal
Compaction	May be needed	Not needed for external fragmentation
Logical view	Strong	Weak
Protection/sharing	Natural per segment	Page-level possible
Virtual memory	Possible	Natural/common

Why Combine Segmentation and Paging?

Segmentation provides:

- logical program organization,
- segment-level protection and sharing.

Paging provides:

- fixed-size physical allocation,
- elimination of external fragmentation,
- convenient virtual-memory support.

Therefore:

Logical segmentation + fixed-size paging

The trade-off is increased address-translation complexity and additional table storage.

Segmentation with Paging: Address Format

A logical address can be represented as:

$$\langle s, p, d \rangle$$

where:

- s = segment number
- p = page number within that segment
- d = page offset

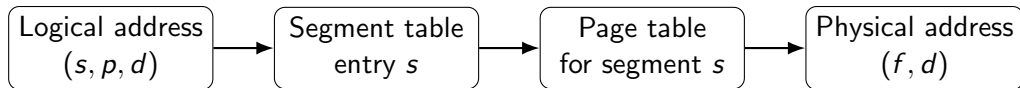
Translation:

$$(s, p, d) \rightarrow \text{segment table} \rightarrow \text{page table for segment } s \rightarrow (f, d)$$

Important

The page tables are associated with segments; the segment number selects the appropriate page table.

Segmentation with Paging: Translation Diagram



$$PA = Frame(p \text{ within } s) \times PageSize + d$$

after checking the segment and page bounds.

Segmentation with Paging: Numerical Example

Given:

- 16-bit logical address.
- 8 segments $\Rightarrow$ 3 segment bits.
- Page size = 256 bytes $\Rightarrow$ 8 offset bits.

Remaining bits for page number:

$$16 - 3 - 8 = \boxed{5}$$

Thus:

$$(s, p, d) = (3 \text{ bits}, 5 \text{ bits}, 8 \text{ bits})$$

Number of pages per segment:

$$2^5 = 32$$

Maximum size of a segment:

$$32 \times 256 = \boxed{8192 \text{ bytes} = 8 \text{ KB}}$$

Segmentation with Paging: Address Translation Example

Assume:

$$(s, p, d) = (2, 5, 100)$$

Segment 2's page table says:

Page 5 $\rightarrow$ *Frame 17*

Page size:

$$1 \text{ KB} = 1024 \text{ bytes}$$

Therefore:

$$PA = 17 \times 1024 + 100$$

$$PA = 17508$$

The segment table first identifies the page table belonging to segment 2; then page 5 is translated to frame 17.

Segmentation with Paging: Advantages

- Retains logical organization of segmentation.
- Supports protection and sharing at segment level.
- Physical allocation uses fixed-size pages/frames.
- Avoids external fragmentation associated with contiguous variable-size segments.
- Works well with virtual-memory mechanisms.

Core idea

Use segmentation for the **logical view** and paging for the **physical allocation**.

Segmentation with Paging: Disadvantages

- More complex address translation.
- Segment table plus page tables consume memory.
- Multiple memory accesses may be required without a TLB.
- Internal fragmentation can still occur because pages are fixed-size.
- TLB/cache mechanisms become important for performance.

More flexibility $\longleftrightarrow$ more translation/storage overhead
--

A Common Source of Confusion

Segmented paging and **paged segmentation** are terms that may be used differently in different sources.

For the scheme used in these slides:

Segment first → page within each segment

Thus:

(s, p, d)

In contrast, a question about a **paged segment table** may mean that the segment table itself is too large and is paged.

GATE tip

Read the exact wording carefully. Do not rely only on the name of the scheme.

Numerical Example: Page-Table Storage

Consider:

$$VA = 32 \text{ bits}, \quad \text{page size} = 4 \text{ KB}, \quad \text{PTE} = 4 \text{ bytes}$$

Offset:

$$\log_2(4096) = 12$$

Virtual page count:

$$2^{32-12} = 2^{20}$$

Page-table storage:

$$2^{20} \times 4 = 2^{22} = \boxed{4 \text{ MB}}$$

Observation

A single-level page table can itself become very large. This motivates hierarchical/multi-level paging.

Numerical Example: Segmentation Table Storage

Suppose a process has:

8 segments

and each segment-table entry is:

16 bytes

Then:

$$\text{Segment table size} = 8 \times 16 = \boxed{128 \text{ bytes}}$$

If 100 processes each have 8 segments:

$$100 \times 128 = \boxed{12800 \text{ bytes}}$$

Comparison

Segmentation table overhead depends primarily on the **number of segments**; page-table overhead depends strongly on the **number of pages**.

Exam-Oriented Comparison

Feature	Segmentation	Paging	Seg.+Paging
Variable-size unit	Yes	No	Segment level
Fixed-size pages	No	Yes	Yes
External fragmentation	Yes	No	No
Internal fragmentation	Low/possible	Yes	Yes
Compaction	Possible	Not required for external frag.	Not required for external frag.
Logical organization	Strong	Weak	Strong
Table overhead	Segment table	Page table	Both
Translation complexity	Moderate	Moderate	High
Protection/sharing	Natural	Page-level	Segment-level + page-level protection

When Would Each Scheme Be Useful?

- **Segmentation:** when logical units, protection domains and sharing are central.
- **Paging:** when efficient physical allocation and virtual memory are central.
- **Segmentation + paging:** when both logical organization and page-based physical management are desired.

Real-world analogy

Library: segmentation = bookshelves by subject; paging = fixed-size boxes used to store book pages. Combining them gives subject-based organization with fixed-size storage units.

Topic: Paged segmentation / segmented paging terminology

A GATE CSE 1995 question asks why, in a particular *paged segmented* memory-management scheme, the segment table itself may require a page table.

Key reasoning:

- A segment table can itself become too large to fit in one page.
- Therefore, the segment table may need to be paged.
- Do not confuse this with paging each segment.

Correct concept: the segment table can itself require paging

Source: GATE CSE 1995, Q1.7.

GATE PYQ 1 – Conceptual Takeaway

The important distinction is:

Situation	What is paged?
Segmented paging	Each segment is divided into pages
Paged segment table	Segment table itself is divided into pages

Exam Strategy

When a question says “segment table itself”, immediately think about the **size of the segment table**, not the size of individual program segments.

Topic: Segmented paging

A GATE CSE 1999 problem considers:

- 2^{16} -byte virtual and physical address spaces,
- 8 equal-sized segments,
- 2-byte page-table entries.

One sub-question asks for the minimum power-of-two page size such that the page table for one segment fits in one page.

Solution:

$$\text{Segment size} = \frac{2^{16}}{8} = 2^{13}$$

Let page size = 2^k bytes.

Number of pages:

$$2^{13-k}$$

Page-table size:

$$2^{13-k} \times 2 = 2^{14-k}$$

GATE PYQ 2 – Further Insight

The same GATE CSE 1999 problem also considers a 512-byte page size and a 16-entry direct-mapped TLB.

For 512-byte pages:

$$512 = 2^9$$

so page offset = 9 bits.

Virtual address = 16 bits:

$$VPN \text{ bits} = 16 - 9 = 7$$

16 TLB entries:

$$\log_2 16 = 4 \text{ index bits}$$

Therefore:

$$TLB \text{ tag bits} = 7 - 4 = 3$$

Exam skill

This question connects **segmentation + paging + page-table sizing + TLB bit calculation**.

Topic: Memory-management overhead comparison

A GATE IT 2006 problem compares the storage overhead for:

- ① paging,
- ② segmentation,
- ③ segmentation with paging

for four concurrently executing processes.

Given values include:

$$\text{Page size} = 1 \text{ KB}, \quad \text{PTE} = 4 \text{ B}, \quad \text{Segment entry} = 8 \text{ B}$$

and different process sizes/segment counts.

Exam method:

- Calculate page-table overhead.
- Calculate segment-table overhead.
- For segmentation with paging, include both segment-table and per-segment page-table overhead.

GATE PYQ 3 – Core Calculation

For the GATE IT 2006 data:

$P_1 = 195$ KB with 4 segments

$P_2 = 254$ KB with 5 segments

$P_3 = 45$ KB with 3 segments

$P_4 = 364$ KB with 8 segments

Segmentation-only overhead:

$$(4 + 5 + 3 + 8) \times 8 = 20 \times 8 = \boxed{160 \text{ B}}$$

For segmentation with paging, each segment can require a page table. The resulting overhead is substantially larger because both the segment-table and page-table structures must be stored.

Important

The exact calculation depends on the paging organization assumed by the question. Follow the stated table/page constraints rather than using a generic formula blindly.

Topic: Two-level paging

32-bit logical address

$$\text{page size} = 4096 = 2^{12} \text{ bytes}$$

$$\text{inner PTE} = 8 = 2^3 \text{ bytes}$$

An inner page table fits in one page, so:

$$\text{entries per inner table} = \frac{2^{12}}{2^3} = 2^9$$

Therefore inner index:

9 bits

Remaining:

$$32 - 12 - 9 = \boxed{11}$$

Thus the outer-page-table index requires:

$\boxed{11 \text{ bits}}$

GATE PYQ Pattern: What You Should Master

From the above PYQs, prepare these patterns:

- 1 Base + offset + limit check in segmentation.
- 2 Page number/offset bit calculation.
- 3 Page-table size calculation.
- 4 Segment-table size calculation.
- 5 Segmentation + paging address fields.
- 6 Minimum page size to contain a page table.
- 7 TLB tag/index/offset calculation.
- 8 Comparing memory-management table overheads.
- 9 Distinguishing “paging a segment” from “paging the segment table”.

Practice Numerical Problem 1

A system uses segmentation.

Segment table:

<i>Segment</i>	<i>Base</i>	<i>Limit</i>
0	4000	1200
1	9000	3000
2	15000	2500

Determine the physical address or fault for:

(0, 1000), (1, 2500), (2, 2600)

Answers:

(0, 1000) $\Rightarrow$ 5000

(1, 2500) $\Rightarrow$ 11500

(2, 2600) $\Rightarrow$ Fault

Practice Numerical Problem 2

A system has:

Logical address = 24 bits

Page size = 1 KB

PTE = 4 bytes

Find:

- ① offset bits,
- ② page-number bits,
- ③ number of virtual pages,
- ④ single-level page-table size.

Solution:

$$\text{offset} = 10$$

$$\text{page number} = 24 - 10 = 14$$

$$\text{pages} = 2^{14}$$

$$\text{PT size} = 2^{14} \times 4 = \boxed{64 \text{ KB}}$$

Practice Numerical Problem 3

A system uses segmentation with paging.

Logical address = 20 bits

Segments = 16

Page size = 256 bytes

Find:

- 1 segment-number bits,
- 2 page-offset bits,
- 3 page-number bits within a segment,
- 4 maximum segment size.

Practice Numerical Problem 3

Solution:

$$s = \log_2 16 = 4$$

$$d = \log_2 256 = 8$$

$$p = 20 - 4 - 8 = 8$$

$$\text{pages/segment} = 2^8$$

$$\text{maximum segment size} = 2^8 \times 2^8 = \boxed{64 \text{ KB}}$$

Practice Numerical Problem 4 – HOT

A process has 6 segments. Each segment is at most 64 KB. Page size is 1 KB and each PTE is 4 bytes.

Tasks:

- 1 Determine the maximum number of pages per segment.
- 2 Determine the page-table size for one maximum-sized segment.
- 3 Determine the total page-table storage for all six maximum-sized segments.
- 4 Explain why the result differs from segmentation-only table storage.

Solution:

$$64/1 = 64 \text{ pages}$$

$$64 \times 4 = \boxed{256 \text{ B per segment}}$$

$$6 \times 256 = \boxed{1536 \text{ B}}$$

The segmentation-only table would store only segment descriptors; segmentation with paging additionally stores page mappings.

Common Exam Mistakes

- Confusing **limit** with the last valid physical address.
- Forgetting the bounds check in segmentation.
- Using logical address bits as physical-address bits without considering frame size.
- Forgetting that page offset bits are determined by page size.
- Calculating page-table entries as $2^{\text{address bits}}$ instead of $2^{\text{page number bits}}$.
- Treating segmentation as fixed-size allocation.
- Claiming paging causes external fragmentation.
- Ignoring page-table storage in overhead comparisons.
- Confusing paging of each segment with paging of the segment table.

Fast Problem-Solving Formula Sheet

Segmentation: $PA = Base_s + Offset, \quad Offset < Limit_s$

Paging: Offset bits = $\log_2(\text{page size})$

Page bits = $VA \text{ bits} - \text{offset bits}$

Pages = $2^{\text{page bits}}$

PT size = $\text{pages} \times \text{PTE size}$

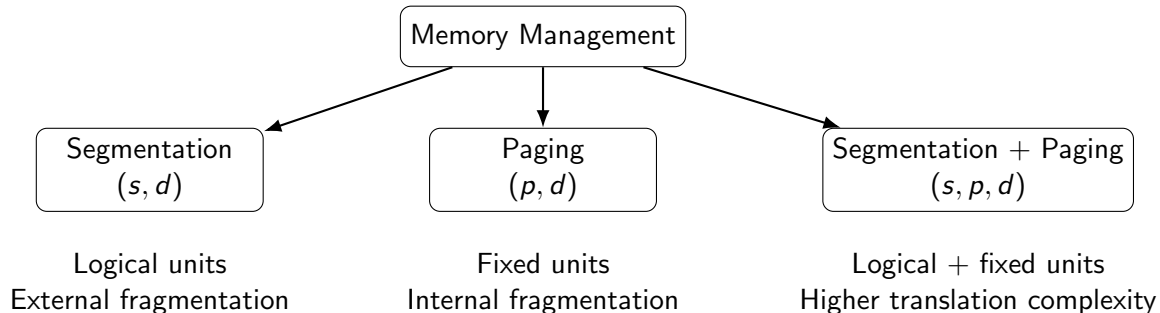
Seg.+Paging: $VA = (s, p, d)$

Segment bits = $\log_2(\text{number of segments})$

Offset bits = $\log_2(\text{page size})$

Page bits = $VA - \text{segment bits} - \text{offset bits}$

Final Concept Map



Summary

- **Segmentation** divides a process into logical, variable-size segments.
- **Paging** divides virtual memory into fixed-size pages and physical memory into frames.
- **Segmentation + paging** combines logical organization with fixed-size allocation.
- Segmentation is associated with **external fragmentation**; paging is associated with **internal fragmentation**.
- Numerical questions frequently test address-bit decomposition, table sizes, bounds checks and translation.
- GATE questions also combine segmentation/paging with TLBs and hierarchical page tables.

Exam focus

Practice the address-translation diagrams and the formula sheet until the bit fields become mechanical.